

***Various Machine Learning Models' Analysis of
Risk Factors for Dog Adoption Longevity***

Submitted by

Matthew Akins
Victor Carrillo
Juna Choi

Abstract:

Many studies have been conducted to predict how fast dogs will be adopted from shelters. Still, there is minimal research that discusses what factors lead to a successful adoption (the dog is never returned to the shelter). In this paper, we use several different classifiers to determine how to best identify dogs that are at a high risk of being returned, before they are adopted. Shelters usually stop tracking data about dogs' adoption the second they leave the door, so the dataset we used is simulated, as the goal of the research is to determine which methods work best in flagging dogs at high risk of adoption before they leave the shelter. We then compared these models to see which features are most strongly associated with returns, focusing on dog behavior and adopter characteristics. Finally, we are using these results to suggest practical ways shelters could adjust counseling and matching processes to reduce returns and support more stable adoptions.

Introduction:

Many shelters have and use data on which dogs get adopted first, and how quickly they get adopted, but not many have data on which adoptions are successful. To solve this problem, we've used a simulated dataset with four different classification models to find the best method of flagging dogs that are at a higher risk of being returned. This is important for two reasons, the first being business implications. Returned dogs directly impact customer satisfaction scores. Second, dogs that have been returned once have a significantly lower chance of being adopted a second time. Being able to flag returns early and take preventative measures could reduce the return rate for high-risk dogs, bringing great benefits for both parties.

The goal of each model is to classify the "returned" variable, which tells us whether or not the dog has been returned to the shelter within 365 days (1 if the dog came back to the shelter, 0 if not). Each model has its own way of selecting variables on which to predict adoption rate; however, "aggression_score" was a strong predictor across all models, with ~59% of returns being behavioral. Mid-to-large-sized adult dogs also seemed to be at higher risk, while puppies and older dogs fared better.

Data Description:

The data is a simulated set using real statistics from dog shelters, since there isn't much actual data on which dogs actually get returned. The overall return rate is set within 7-20% of all adoptions, and dog behavior is the leading predictor of returns at ~59%. Other important variables are set within reasonable ranges, such as size, owner-related reasons (home size, whether the family has kids or already has pets, or if the owner has a backyard), and adopter expectations.

The dataset consists of 32 columns, of which 28 are usable predictors, with 42,000 observations (adoptions). Some of the most important variables included: Size (small, medium, large, and XL), weight (kg), age in years, days in shelter, previously returned (1 if yes), household has kids/pets (1 if yes), house trained (1 if yes), expectation score (0-10), size and home mismatch (1 if a large dog is going to an apartment), and adoption counseling (1 if dog received adoption counseling).

Exploratory Data Analysis:

The first step is to catch leakage on any individual features. To do this, we run ROC-AUC on every feature to see which performs best. The result is that “days_to_return” and “return_reason” had perfect scores (Figure 1), a huge gap between those and the next best predictor, “aggression_score”, ~0.66. “Days_to_return” tells us how long it took the dog to be returned to the shelter, while “return_reason” gives the reason why the dog was returned. Both columns remain empty if the dog wasn’t returned. Leaving these variables for the model to use as predictors would allow the model to cheat off them, giving false, high accuracy results. In addition, we drop “adoption_id” as each observation gets its own unique value, which would add nothing to the models.

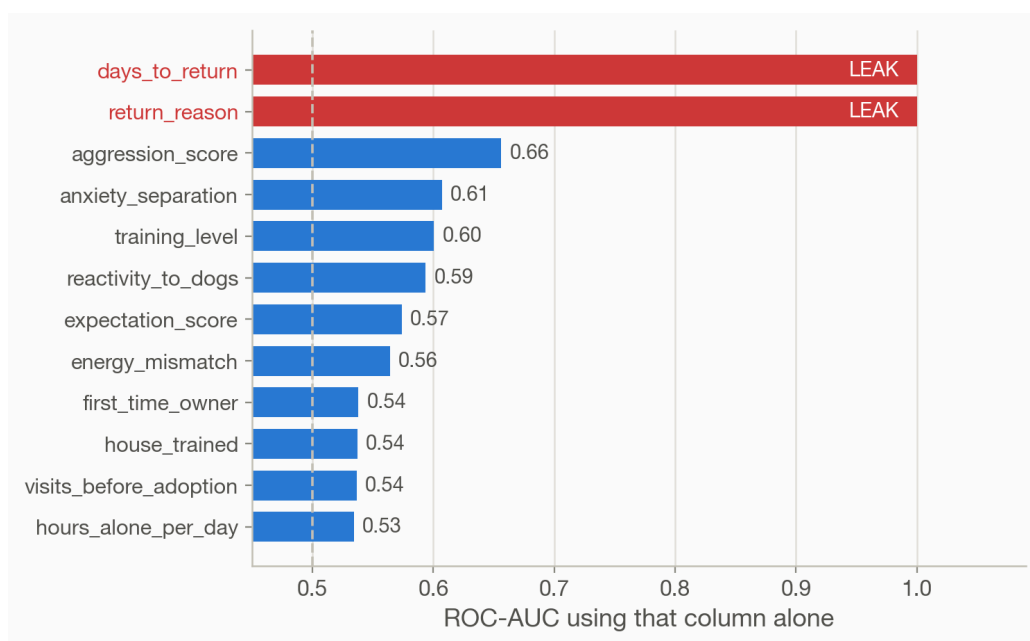


Figure 1. Each column scored on its own with a shallow decision tree. `days_to_return` and `return_reason` reach a perfect 1.000 because both are only filled in after a return has happened. Both were dropped.

Another important facet of this dataset is that we cannot use accuracy for any of the models. Only 15% of dogs are returned, so a model that guesses 100% kept would already be 85% accurate (Figure 2). For this reason, we use other methods of measuring how well our models perform, such as ROC-AUC. Looking ahead to the models themselves, the return rate

rises steadily across the aggression score, from 8% in the lowest band to 46% in the highest (Figure 3).

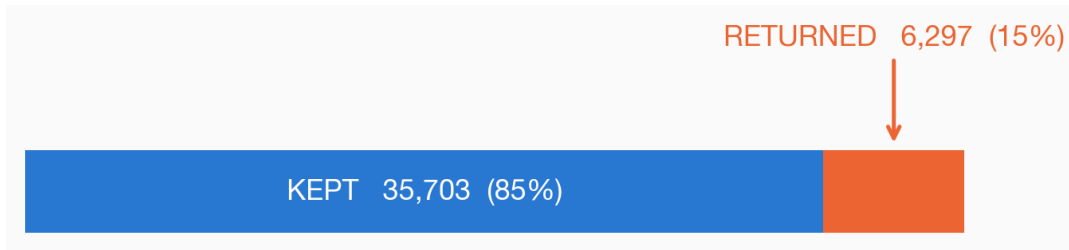


Figure 2. The class balance. Only 15% of adoptions end in a return, so a model that always predicts kept is already 85% accurate while catching nothing.

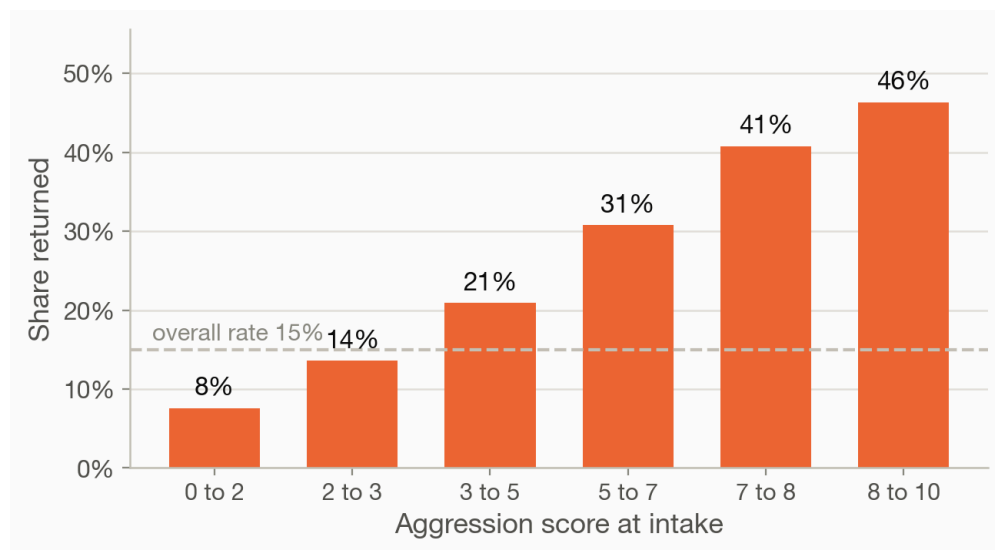


Figure 3. Return rate across the aggression score recorded at intake. The rate climbs from 8% to 46%, which is the largest single effect in the data.

Statistical Methodology:

To achieve our goal of predicting which dogs will be returned within 365 days, we aim to find the best classifier that can accurately make this prediction. We chose among four different classifiers for this: K-Nearest Neighbors, Multi-layered Perceptron, Decision Trees, and Nearest Centroid. For every model, we used the Scikit-Learn Python library.

For all models except for the decision tree, feature scaling was necessary. Some features, such as “days_in_shelter,” span from 1 to 350, while others like “aggression_score” only span from 0 to 10. These features need to be weighted equally so that the model doesn’t overvalue one feature over another. Each feature is scaled according to its distance from the mean. Non-numerical columns such as “home_type” (apt, rental, house) get split into binary (yes or no) columns, and coded as 1s or 0s, called one-hot encoding, which adds 14 columns to

the set. Ordinal columns such as “size” are also encoded, but aren’t split into new columns. This way, each feature gets equal representation.

Before beginning, we want to make sure each model is trained and tested on the same data, so that we can accurately compare their performance. We used 75% of the data for training and 25% for testing. Both groups retain the ~15% return rate. This makes sure that none of the models get a “lucky draw” on the data and perform better than others (Figure 4).

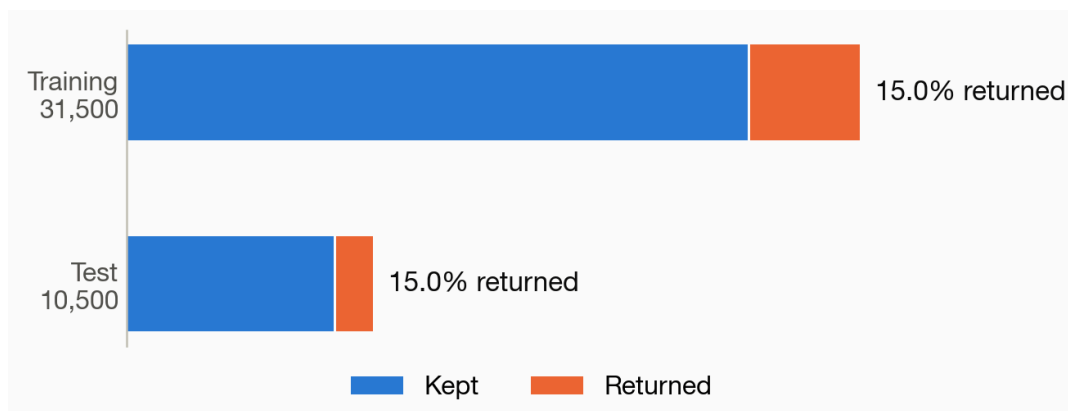


Figure 4. The stratified 75/25 split. Both halves carry the same 15% return rate, and every model was trained and tested on exactly these rows.

Nearest Centroid:

The nearest centroid classifier builds a profile of each outcome class by averaging every feature across the kept adoptions and, separately, across the returned adoptions, then assigns each new case to whichever profile is nearer in Euclidean distance; the fitted model consists entirely of these two class centroids, one mean per feature per class. We trained it on 42,000 simulated adoption records, of which roughly 15% ended in a return, using 39 features covering dog behavior, adopter characteristics, and dog-adopter compatibility. Because the standard implementation returns only a hard binary prediction, we wrapped the classifier to expose the signed gap between the two distances, which reproduces the built-in rule exactly while allowing evaluation on a ROC curve alongside the other models. We standardized the full feature matrix, applied the same stratified 25% test split and cross-validation folds used throughout the study, selected the shrinkage threshold by cross-validation (0.571, reducing the contributing features from 39 to 27, Figure 5), and moved the decision cut-off to 0.18, since the default nearer-centroid rule assumes equally common classes when kept adoptions outnumber returns roughly six to one (Figures 6 and 7). On held-out data the tuned model reached a ROC-AUC of 0.7544, identifying 985 of the 1,574 true returns (recall 0.626) at a precision of 0.304, satisfying the study's 0.30 precision floor (Figure 8). The model depends most heavily on aggression score, followed by separation anxiety, reactivity toward other dogs, training level, adopter expectations, and energy mismatch (Figure 9), indicating that dog behavior and dog-adopter compatibility, rather than demographics like breed, size, or home type, are the strongest signals shelters could act on before an adoption is finalized.

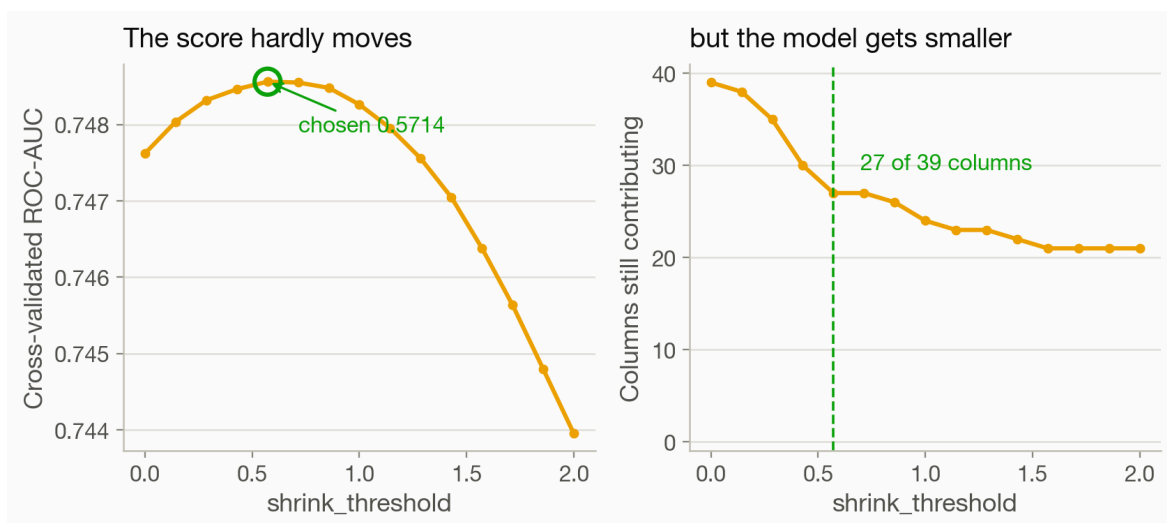


Figure 5. Cross-validation of the shrinkage threshold. The score moves very little across the whole sweep, but the chosen value of 0.571 reduces the contributing columns from 39 to 27.

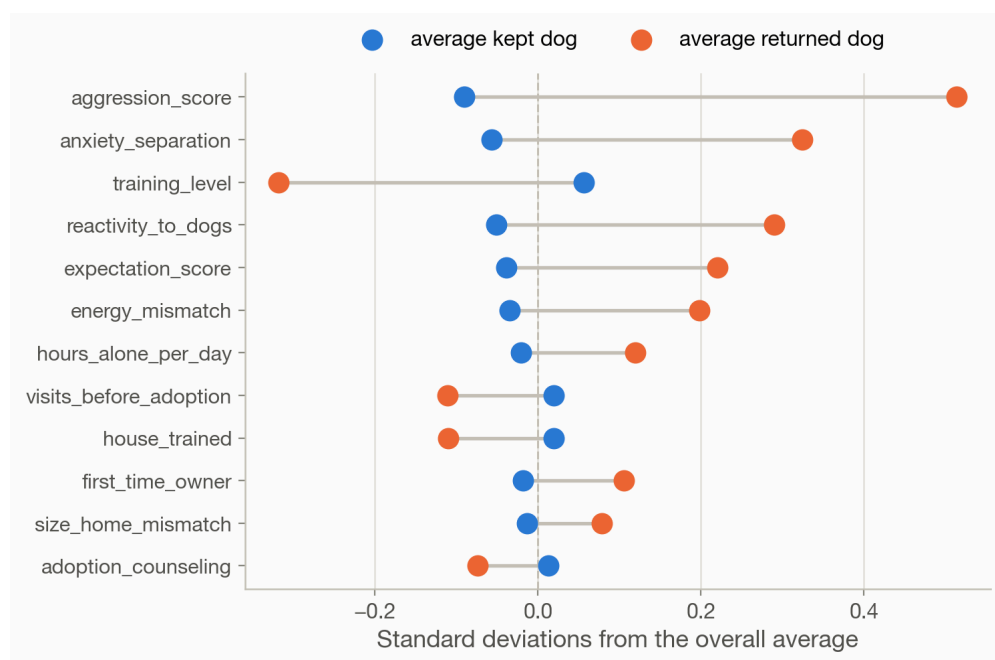


Figure 6. The fitted model in full. Each row shows one column's average for kept adoptions and for returned adoptions, in standard deviations from the overall mean.

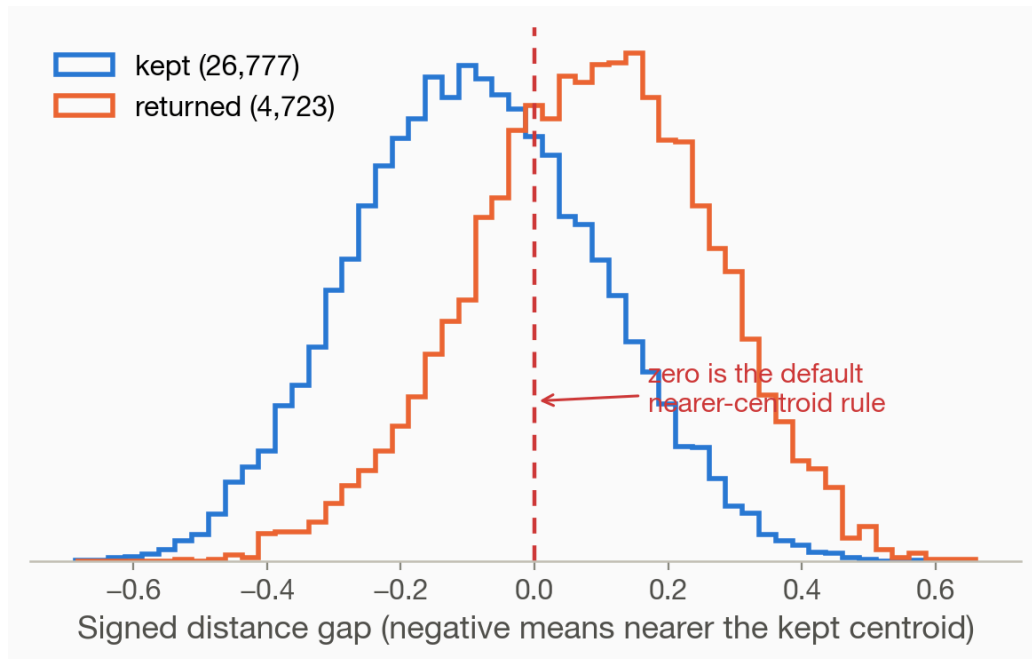


Figure 7. The signed distance gap on the training data, split by outcome. The default nearer-centroid rule cuts at zero, which assumes the two classes are equally common.

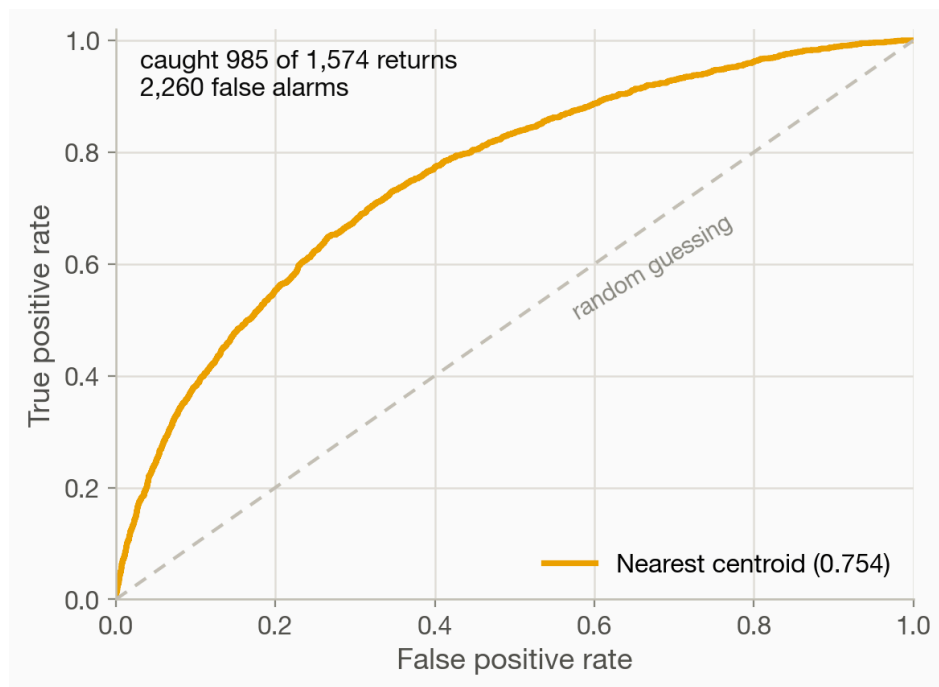


Figure 8. Nearest centroid on held-out data. ROC-AUC 0.7544, identifying 985 of the 1,574 true returns at a precision of 0.304.

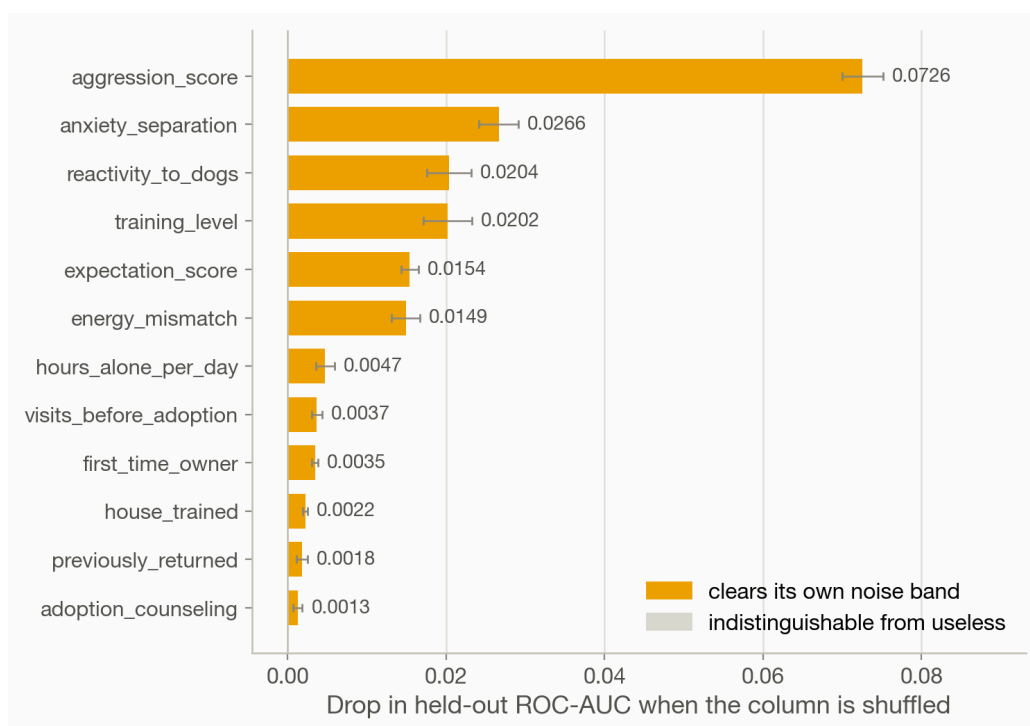


Figure 9. Permutation importance for nearest centroid, measured as the drop in held-out ROC-AUC when a column is shuffled.

Multi-Layered Perceptron:

For multi-layered perceptrons, the main feature of the classifier is the activation function, otherwise known as ReLU. ReLU determines the importance of each neuron in multilayered functions. If the neuron returns a negative value, the ReLU function returns 0, as it ignores any neuron that does not contribute positively to the analysis. It only takes positive correlations because negative ones do not meaningfully contribute to the overall model. This allows for different neurons to specialize in different areas by "activating" relevant neurons. Figure 10 demonstrates this, where values below 0 are calibrated to be 0.

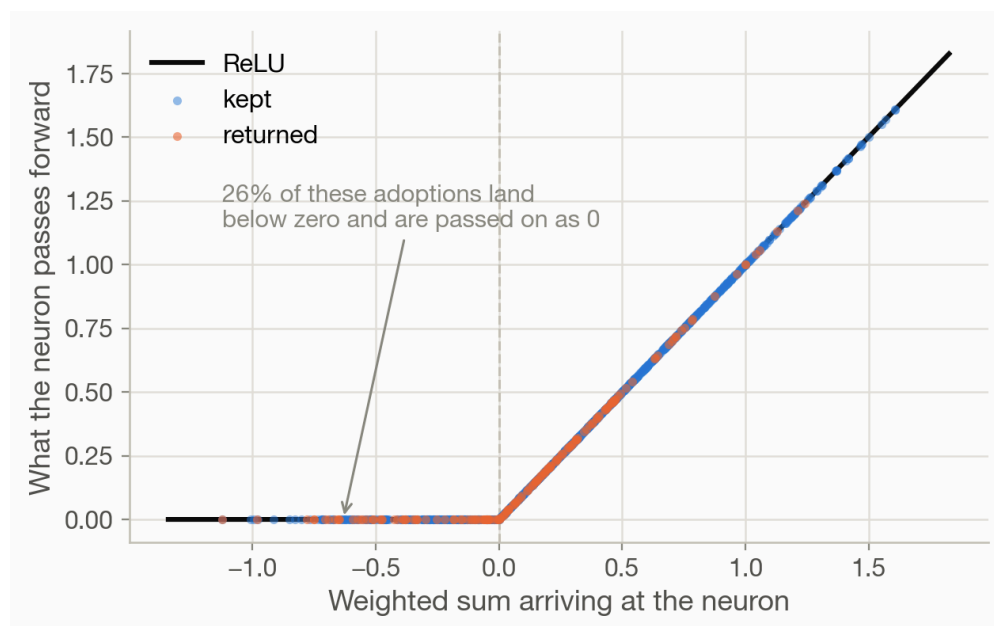


Figure 10. The ReLU activation, drawn with the pre-activation values of 1,200 real training adoptions plotted on it. Any negative weighted sum is passed forward as zero, so a neuron contributes only when its input is positive.

For the final output neuron, a sigmoid function is used instead of ReLU. As shown in Figure 11, a sigmoid pushes all negative inputs towards 0, positive inputs towards 1, and inputs near 0 towards .5.

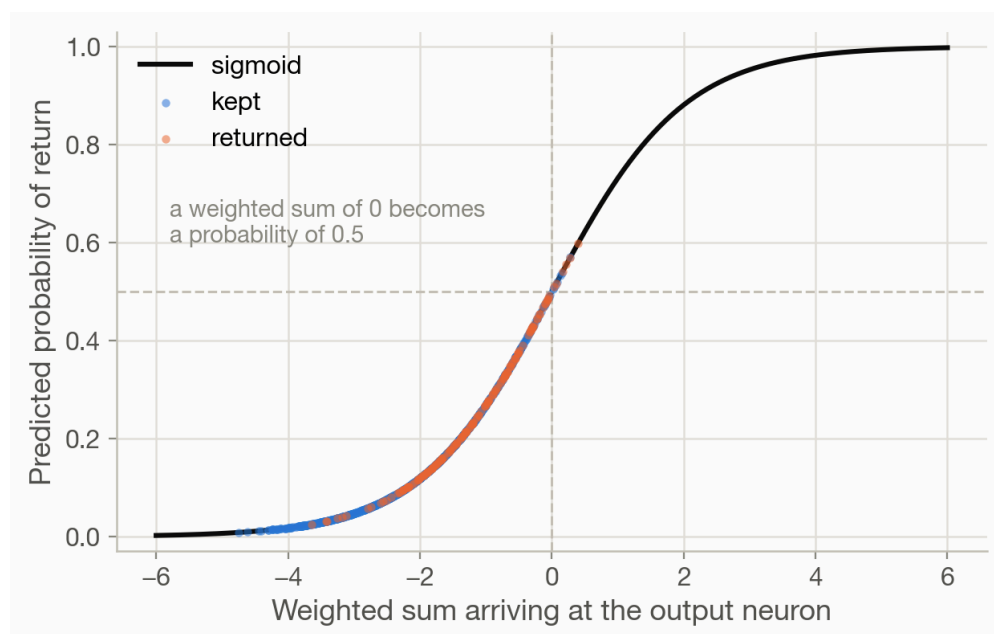


Figure 11. The sigmoid used at the output neuron, again with real training adoptions plotted on the curve. It squashes any weighted sum into a value between 0 and 1, which is what turns the network's output into a predicted probability of return.

This results in an actual percentage of returned values. This model has one hidden layer with 32 neurons. This architecture was decided with logic. A sweep test was done that tested a single layer of 32 neurons, a single layer of 64, a two-layer setup of 64 and 32, and a two-layer setup of 128 and 64. The single hidden layer with 32 neurons produced the best cross-validated ROC-AUC, 0.7084. Then, a second sweep test for alpha values showed that $\alpha = 1$ was the best and further increased the ROC-AUC to 0.7524. Finally, the model was retrained on all 31,500 training rows. The model stopped improving after 63 passes as that is when the loss stopped improving. In the end there were 1,313 total weights and the final training loss was 0.3709. The number of neurons is a good fit for our data size of 42,000, as if the number of neurons is too small, then the patterns of the dataset will not be fully encapsulated, and if the number of neurons is too large, then the model becomes too specific and won't be able to predict accurately on new test data. The model makes predictions on the training set, and then calculates the loss of these predictions. Then, the model is trained using gradient descent to calculate the slopes of all the losses with respect to all the weights. Figure 12 shows the process of minimizing the losses through gradient descent by taking the slopes.

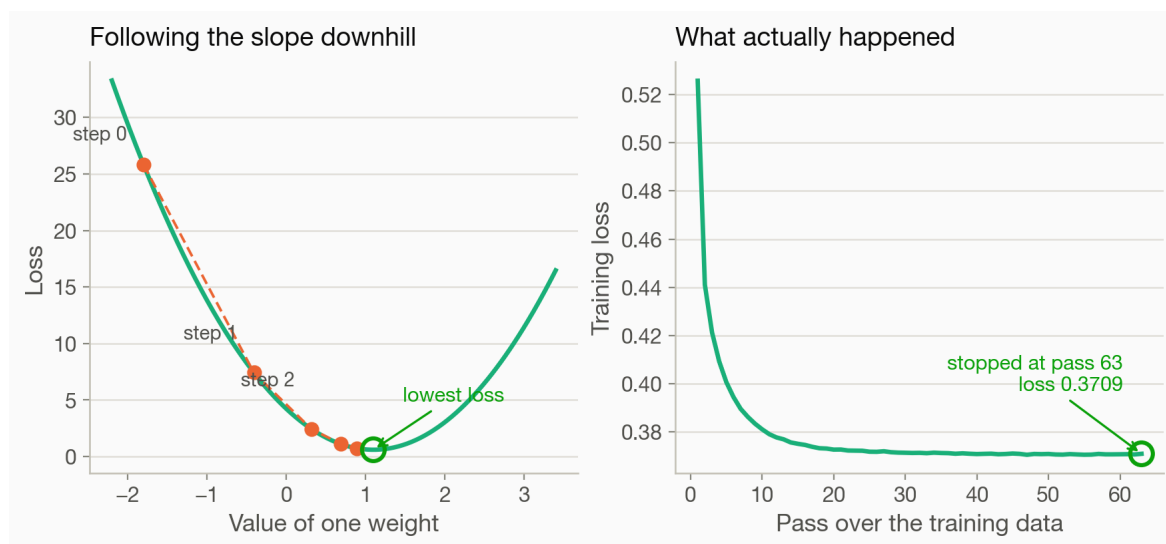


Figure 12. Gradient descent. On the left, the idea: the model measures the slope of the loss with respect to a weight and steps downhill. On the right, what actually happened during training, with the loss flattening and training stopping after 63 passes.

This gradient descent uses backpropagation. The model does this in order to minimize the loss and reduce inaccurate categorization. The accuracy from the multi-layered perceptron is 73.5%. Although this may seem better than standard guessing, the baseline amount of dogs that are "kept" is, as mentioned previously, 85%. This means that the multi-layered perceptron has worse accuracy than the baseline data. However, this is expected because there is a trade-off between accuracy and recall on the rare class. Furthermore, the model produced a recall of .64, and a precision of .31. This means that the model flags 64% of the dogs that were actually returned, and of the dogs labeled returned, only 31% of them were returned. This means that the multi-layer perceptron is able to flag high-risk dogs rather than identify with accuracy whether a dog will be returned. The precision and recall values' importance depends

on the cutoff of the risk. This cutoff value was decided through cross-validation on the training data and was the lowest threshold that keeps precision above 0.30. This cutoff value was found to be 0.179. This cut off is crucial in counteracting the 15% class imbalance. Finally, the perceptron flagged aggression_score as the most important factor, with the largest drop in ROC-AUC, 0.07597, when that column is shuffled (Figure 13), which matches with the other three classifiers mentioned in this paper.

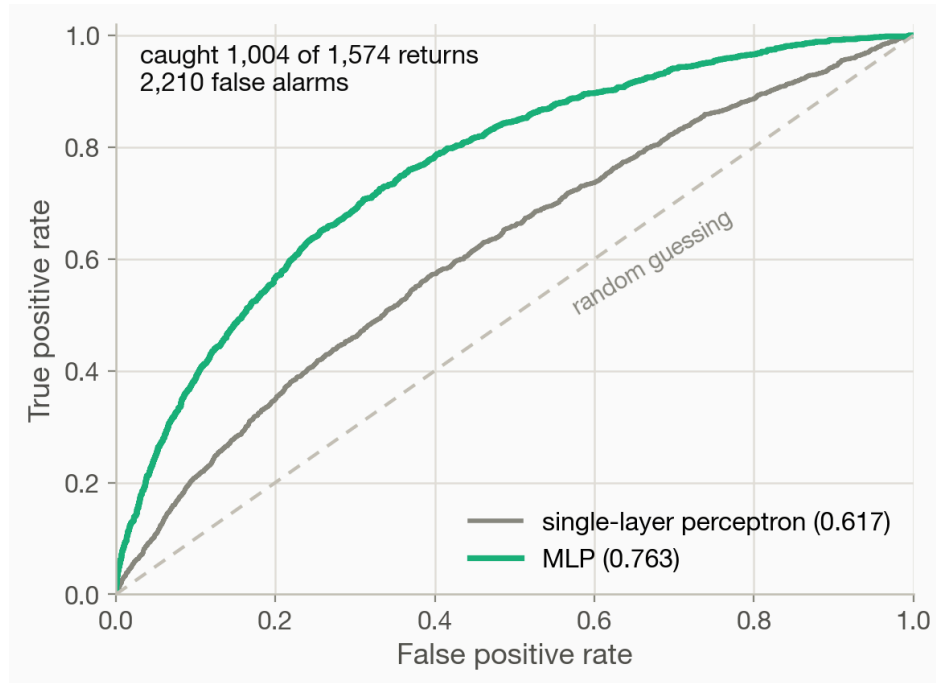


Figure 13. The multi-layered perceptron on held-out data, against the single-layer perceptron it replaces. The hidden layer raises ROC-AUC from 0.617 to 0.7626.

K-Nearest Neighbors Classifier:

K-Nearest Neighbors is considered a lazy learner, as there is almost no training step. The raw 31,500 observations that the model should train on are the model itself. The first step for K-Nearest Neighbors is defining what “near” means. For each new adoption, it subtracts its value from the stored value in each column, squares those differences, sums them over the 39 columns, and takes the square root. This is the distance, and the reason why the previous scaling step was so important. It then takes the k closest adoptions using those distances, and weights them $1/\text{distance}$, so that closer dogs count more.

It chooses K through cross-validation, choosing ten different values from 11 to 6401. The score climbs quickly, then plateaus. While the best score was a K of 6401, the last two

doublings only added ~0.001, so anything above 1601 performs about the same. This tells us that the signal is weak (Figure 14).

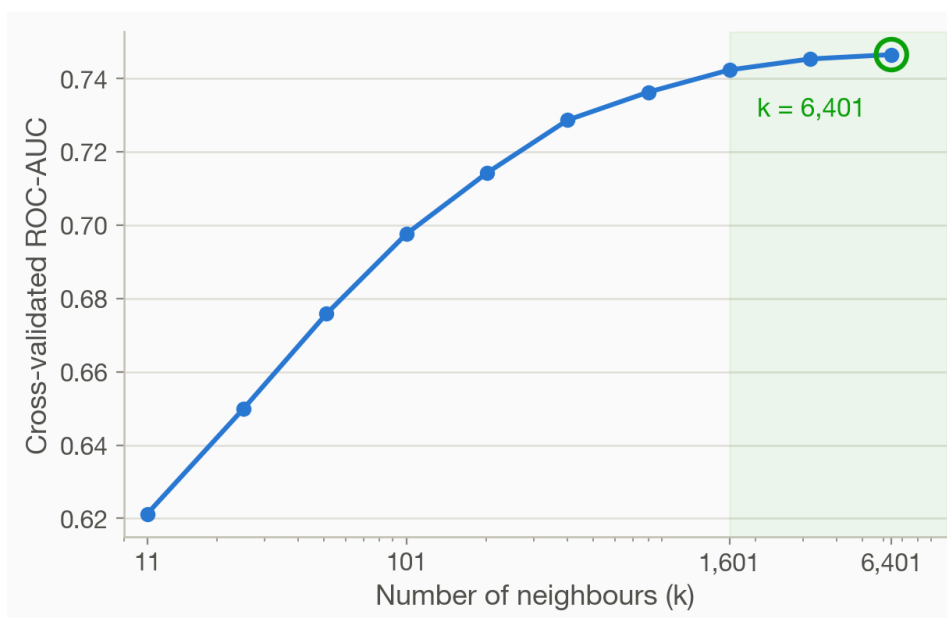


Figure 14. Cross-validated ROC-AUC across ten values of k . The score climbs quickly and then plateaus, so any value above about 1,601 performs the same.

In a normal KNN classifier, we'd use a cutoff of 0.5, meaning the majority neighbors win. In this case, since the return rate is ~15%, a 0.5 cutoff would fail badly; it would flag no adoptions at all. Instead, we choose the cutoff by cross-validation as the lowest threshold that keeps precision above 0.30, which gives 0.149. This buys as much recall, the share of dogs that came back that were flagged, as we can get while holding precision, the share of flagged dogs that came back, at an acceptable level.

The results were promising; the model caught 1,017 returns out of 1,574, with 2,339 false alarms (Figure 15). The most important factors were "aggression_score", "anxiety_separation", and "energy_mismatch". These results tell us that a K-Nearest Neighbors model would be good enough to give warnings and follow-up calls, but not enough to warrant a denied adoption.

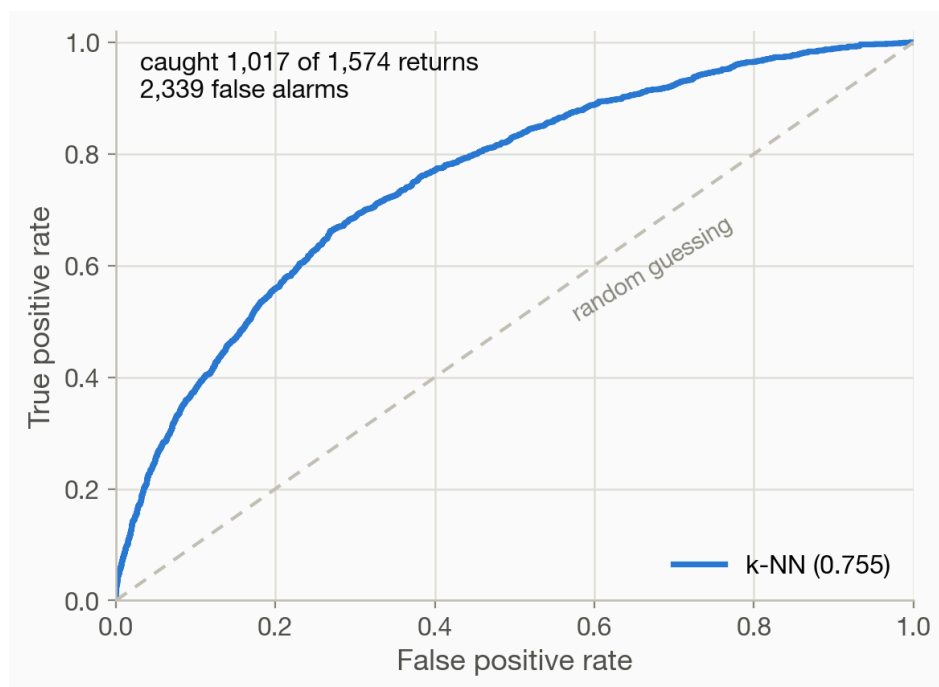


Figure 15. K-Nearest Neighbors on held-out data. ROC-AUC 0.7548, catching 1,017 of the 1,574 returns with 2,339 false alarms.

Decision Tree:

The first step for the decision tree is to find which first question best separates returns from keeps. Asking if the adopted dog had an aggression score of ≤ 2.55 worked best. Each branch does this again recursively, creating new branches and repeating. The model then rebalances its classes; the goal of the model is to catch returned dogs, so a return weighs about 5.7 times as much as a keep (Figure 16). This is why the cutoff for this model is closer to 0.5, different from other models.

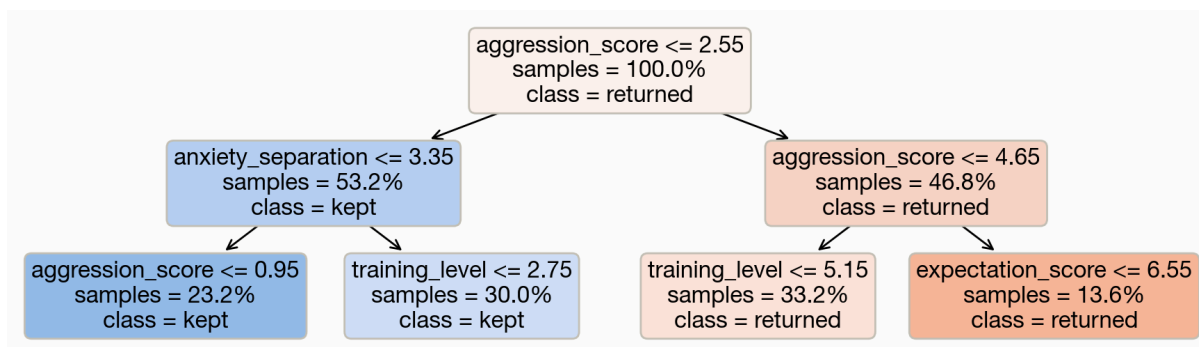


Figure 16. The top two levels of the pruned tree. The first question asked is whether the aggression score is 2.55 or lower.

The next step is pruning; without this step, the model would continue recursively until every branch was completely pure, which would be too complex for our needs without offering a substantial reward. Instead, we remove branches that do not add to the accuracy of the model. To do this, we charge a fee of α (alpha). A larger α makes the tree smaller because the cost of more branches is high, and a smaller α allows the tree to grow larger with more branches. We chose our alpha through cross-validation, finding the peak of ROC-AUC. This resulted in an α of about 0.00066, giving us a tree with 26 leaves and a depth of 7 (Figure 17).

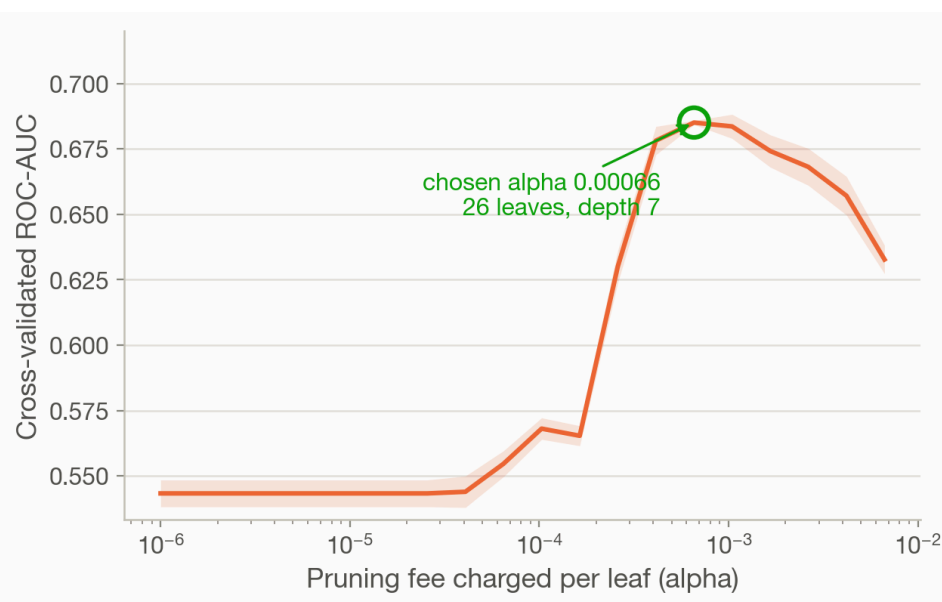


Figure 17. Cross-validated ROC-AUC against the pruning fee. The peak sits at an alpha of about 0.00066, giving a tree with 26 leaves and a depth of 7.

The result was not as promising as other models; it caught only 530 out of the 1,574 returns at the cost of 1,149 false alarms (Figure 18), about half of what other models were able to do. However, similarly to other models, it flagged “aggression_score” as a high predictor of returns. It also flagged “anxiety_separation” and “expectation_score” as high return predictors. This model could use work, as its results fall short of what others were able to produce.

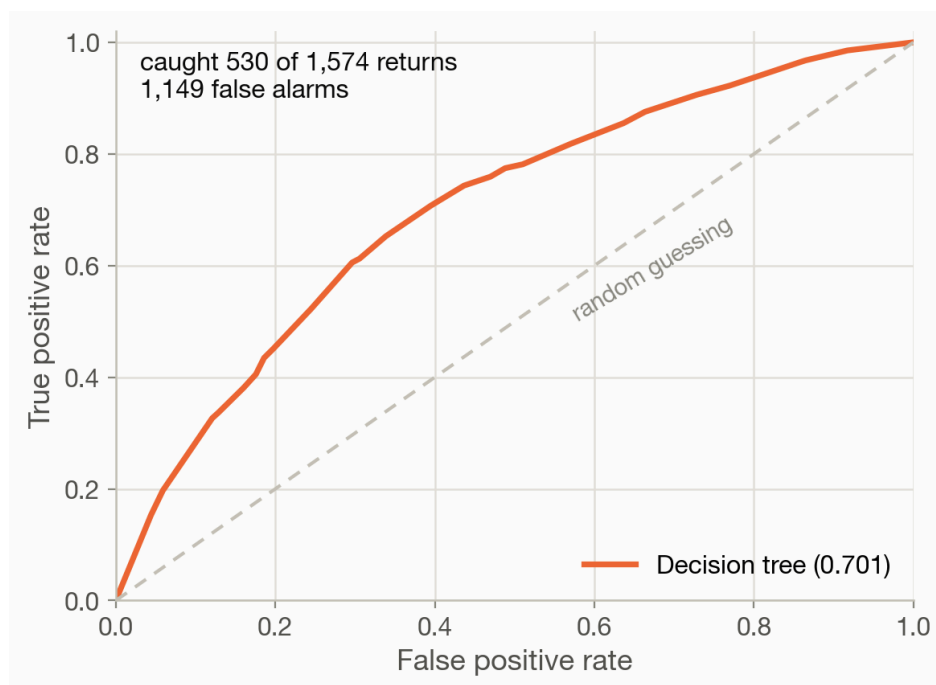


Figure 18. The decision tree on held-out data. ROC-AUC 0.7005, catching 530 of the 1,574 returns with 1,149 false alarms.

Comparisons Between Classifiers:

When comparing Nearest Centroid, MLP, KNN, and Decision Tree to one another in different areas, there are a few key findings that determine their usefulness. The Decision Tree is the one that guesses the most accurately among all the classifiers. This does not mean that it is necessarily a better classifier. The reason for this is that the cut-off for this classifier is substantially higher, and therefore stricter, than the other 3 classifiers. Additionally, similar to our baseline guessing method of assuming all are “kept”, groups within the Decision Tree are heavily populated with dogs that are kept and not returned. This means that as long as a group does not happen to have many returned dogs, that group will be labeled as kept, which matches the actual dataset’s outcomes. However, this does not necessarily advocate for the validity of the test, as it does not produce meaningful results about the actual weights of the reasons for the returns on the dogs. When comparing Nearest Centroid, Decision Trees, and KNN to the MLP’s ROC-AUC, we find that MLP has a better ROC-AUC than the rest of the classifiers. This means that out of all the classifiers, MLP is better at ranking which dogs are higher-risk relative to one another. We can conclude that the multilayer perceptron performed the best out of the four models, with an ROC-AUC of 0.7626 (Figure 19).

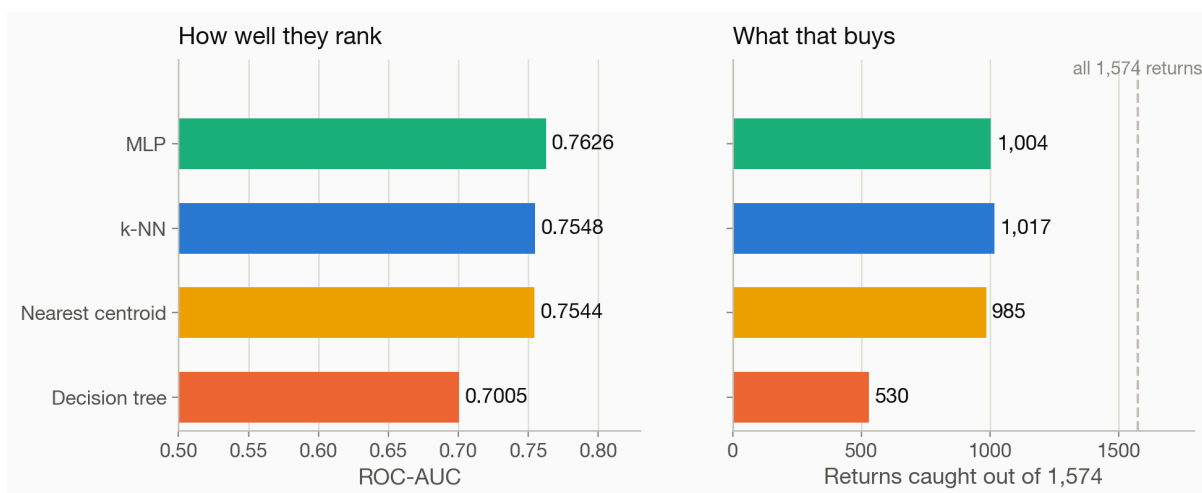


Figure 19. All four classifiers on the same held-out data. The multi-layered perceptron ranks highest, though the top three sit within 0.009 of one another, and K-Nearest Neighbors catches the most returns.

Although MLP does have a substantial edge over the other classifiers in terms of the highest ROC-AUC, when looking at the real-life applications of these models, the other models also have their own strengths.

Nearest Centroid and KNN performed rather similarly, scoring an ROC-AUC within 0.0004 of each other. In terms of speed, KNN is the slowest out of all the models, and becomes increasingly slower over time. However, KNN caught more returned dogs than the MLP, which caught 1,004, while KNN caught 1,017. KNN also has the second-best ROC-AUC out of all of the models. Nearest Centroid and Decision Tree are the two fastest models and are able to calculate the results at a fraction of the time it takes to calculate the results for MLP and KNN. The ROC-AUC of Nearest Centroid is also only .0082 away from the ROC-AUC of MLP. Although this is indeed better for MLP, MLP also has other caveats such as a delicate architecture. For Nearest Centroid, there are no such concerns about noise memorization.

It can be concluded that for statistical analysis, MLP has the clear advantage with the highest ROC-AUC. However, because Nearest Centroid is faster, has no noise memorization, and has a neglectful difference in ROC-AUC with MLP, there is a clear argument for the usage of Nearest Centroid in an actual dog adoption facility.

References

Nefedov, S. (2026). Dog adoption success — Does the match last? [Data set]. Kaggle.
<https://www.kaggle.com/datasets/sergionefedov/dog-adoption-success-does-the-match-last>